CompMem: Compressive Memory for Long-Term AI Chatbots

CompMem keeps about 80–86% of full-context answer quality (F1 0.25–0.27 vs 0.31) with about 65% fewer prompt tokens, and cuts the memory needed at 100k tokens from 1.2 GB to under 2 MB.

Venushree Gayathri C U, Manipal Institute of Technology, Bengaluru. IDEAS-TIH Autumn Internship 2026, Indian Statistical Institute, Kolkata. Mentor: Dr. Chandan Biswas.

Abstract

Long chatbot conversations slow standard LLMs quadratically and fill GPU memory with KV-cache. CompMem turns each chunk of at most 256 tokens into a 64-bit barcode of -1 and +1, retrieves relevant chunks with Multi-Index Hashing, feeds only those chunks to the LLM with a soft attention bias, and keeps memory bounded with a background cleaner.

On HotpotQA with Qwen2.5-0.5B-Instruct, CompMem retrieves both supporting paragraphs in the top 4 for 43% of questions (random: 5%) and reaches F1 of 0.25 to 0.27 against 0.31 for full context, using about 65% fewer prompt tokens. At 100k tokens it needs 1.8 MB instead of 1.2 GB of KV-cache. Training the barcode maker end-to-end through the LLM's attention bias lowered the held-out answer loss from 2.217 to 2.049 and raised retrieval from 0.38 to 0.43.

Problem and approach

Standard attention compares every new token with every past token, so compute grows as O(N^2) and the KV-cache grows linearly with the conversation, reaching gigabytes for long chats. CompMem replaces the full history with compact memories:

1. Cut the conversation into chunks of at most 256 tokens.
2. Turn each chunk into a 64-bit barcode of -1 and +1 with a learnable hashing encoder.
3. For a new question, find the chunks whose barcodes are within 7 bits of the question's barcode and give only those to the LLM.
4. Scale the LLM's attention to those chunks by how well they match and how useful they are, and clean old memories in the background.

System design

Phase	Component	Implementation
1	Barcode maker	Chunker (at most 256 tokens); frozen MiniLM Transformer plus a trainable hash head; logistic noise, tanh, then sign, with the straight-through estimator h = sign(h~) + (h~ - h~.detach()) so gradients pass through the sign step
2	Search index	Multi-Index Hashing with 4 tables of 16-bit pieces. If two codes differ in at most 7 bits, one piece differs in at most 1 bit (pigeonhole), so probing the exact piece and its 16 one-bit flips is exact; it matched brute force on 400 queries. Chunks within 4 bits of a cluster medoid join that cluster, which keeps a running average barcode
3	Attention connector	Match score alpha = (1 + h~_q . h_i / 64) / 2; keep score r = sigmoid(W . mu + b) from the cluster's average barcode; the bias log(alpha * r + 1e-6) is added to the attention scores of the retrieved tokens in every layer and head through a custom attention mask
4	Memory cleaner	Asynchronous; above 512 chunks it deletes clusters with r below 0.25 and asks the LLM to summarize clusters with more than 5 chunks, then re-hashes the summary

Because alpha * r is at most 1, the bias is never positive: it suppresses weak or noisy chunks and leaves a perfect, useful match unchanged.

Experimental setup

- **Data:** HotpotQA (distractor setting), dev split of 7,405 questions: 3,000 for training and 500 held out. Each question's memory is its 10 paragraphs (2 supporting, 8 distractors) plus 6 small-talk chunks, 16 chunks in total; 4 chunks are retrieved. Random chance of getting both supporting paragraphs into the top 4 is 5%.

- **Encoder:** frozen all-MiniLM-L6-v2, then an MLP hash head (512 hidden units) producing 64 bits.
- **LLM:** Qwen2.5-0.5B-Instruct, frozen. Answer quality (F1, exact match) is measured on 100 held-out questions on a Google Colab T4 GPU.
- **Stage 1 training:** 30 epochs with an InfoNCE retrieval loss on supporting-fact labels and a bit-balance penalty; the keep layer is trained on supporting versus other chunks with the lambda * sum(r) penalty.
- **Stage 2 (end-to-end) training:** 1,200 episodes. The loss is the cross-entropy of the gold answer through the LLM with the attention bias, plus 0.01 * sum(r) and a small retrieval loss. Gradients flow through the frozen LLM into alpha (hash head) and r (keep layer). Learning rate 1e-4.
- **Benchmark:** Qwen2.5-0.5B dimensions (24 layers, 14 heads, 2 KV heads, head size 64). Standard attention time is measured on the GPU up to 32k tokens and extrapolated beyond with a quadratic fit; CompMem is measured at every length; KV-cache size is computed from the model configuration.

Results

Retrieval and answer quality

The 64-bit codes retrieve both supporting paragraphs in the top 4 for 38% of the 500 test questions after stage 1 and 43% after end-to-end training, against 5% for random choice (brute-force Hamming ranking). Through the Multi-Index Hashing store, stage 1 scored 40%.

Setup (100 held-out questions)	F1	Exact match	Prompt tokens
Closed book (no memory)	0.069	0.03	62
4 random chunks	0.159	0.12	422
Full context (standard LLM, 10 paragraphs)	0.308	0.24	1494
CompMem, stage 1, no modulation	0.256	0.19	521
CompMem, stage 1, with modulation	0.265	0.19	521
CompMem, end-to-end, no modulation	0.261	0.20	544
CompMem, end-to-end, with modulation	0.247	0.20	544

Setup (100 held-out questions)	F1	Exact match	Prompt tokens
Oracle (supporting paragraphs only)	0.536	0.44	295

CompMem keeps 80 to 86% of full-context F1 with about 65% fewer prompt tokens. With 100 questions the F1 standard error is about 0.04, so the differences between the four CompMem variants are within noise and no F1 gain from modulation is claimed. The gap to the oracle shows that retrieval quality is the main bottleneck.

Effect of end-to-end training

Held-out measure	Stage 1	End-to-end
Answer cross-entropy with modulation (lower is better)	2.217	2.049
Answer cross-entropy without modulation	2.137	2.137
Both supporting paragraphs retrieved (100 questions)	0.39	0.46

Before end-to-end training, the attention bias made the answer loss worse than using no bias (2.217 against 2.137); afterwards it lowers it (2.049). The gradient of the answer loss reaching the hash head was non-zero throughout training, confirming that the answer loss trains the barcode maker.

Keep score and cleaner

The stage 1 keep score separates small talk well but supporting paragraphs from distractors only weakly: mean r is 0.03 for small talk, 0.23 for distractors and 0.30 for supporting paragraphs (AUC 0.78 for supporting versus all others). In the cleaner test, 620 chunks (300 noise chunks and 40 clusters of 8 near-duplicates) were reduced to 40: all 300 noise chunks were deleted and the 40 crowded clusters were each summarized. With the real LLM, six repeated notes about an exam became one sentence: "The exam is scheduled for Friday, and it's important to remember to review your graphs before the exam."

Memory and speed

Conversation length (tokens)	Standard KV-cache	CompMem memory	Standard attention time	CompMem time
8,000	98.3 MB	0.14 MB	0.22 s	0.08 s
32,000	393.2 MB	0.56 MB	3.67 s	0.33 s
64,000	786.4 MB	1.12 MB	14.87 s (extrapolated)	0.66 s
100,000	1228.8 MB	1.82 MB	36.47 s (extrapolated)	1.03 s

Standard time is measured up to 32k tokens and extrapolated beyond. CompMem memory counts the barcodes, the index and the chunk text, not the model weights. The code repository contains the plots training_curves.png, e2e_curves.png and benchmark.png.

Deviations from the SOW and limitations

- **Encoder:** a frozen pretrained MiniLM Transformer feeds a trainable hash head, instead of training a Transformer from scratch, which needs far more data than was available.
- **Extra losses:** a small InfoNCE retrieval loss and a bit-balance penalty are added, because the lookup step is not differentiable and the answer loss alone gives the encoder no direct retrieval signal. They are kept small in the end-to-end stage.
- **Keep layer:** stage 1 trains it with supporting-fact labels; the end-to-end stage adds the answer loss and the lambda * sum(r) penalty as described in the SOW.
- **Benchmark:** the 100k-token numbers come from a script that measures the components on synthetic text. It is not a live chatbot run on a real 100k-token conversation, and standard-LLM time beyond 32k tokens is extrapolated.
- **Scale of evaluation:** 100 questions for answer quality, a 0.5B-parameter LLM, and short HotpotQA memories (16 chunks), so the results are indicative rather than a full benchmark.
- **Training:** stage 1 test recall peaked near epoch 14 and then drifted down while training loss kept falling, a mild overfit that early stopping would reduce.

Conclusion and future work

All four phases of CompMem are implemented and tested: the barcode maker, the hash-table search, the attention connector trained end-to-end through a real LLM, and the background cleaner. The system keeps most of full-context answer quality while using about a third of the prompt tokens and a memory footprint that stays under 2 MB at 100k tokens. Retrieval quality is the main limit on accuracy.

Next steps:

- Run a live chatbot on real long conversations of up to 100k tokens, with the cleaner and the LLM summarizer working together.
- Improve retrieval with early stopping, more training data and a larger or fine-tuned encoder.
- Evaluate with a larger LLM and more questions, and with multi-hop datasets beyond HotpotQA.
- Train the keep score on usage signals so it can tell useful content from distractors, not only from small talk.